

AI Scientist (공통) 기술 역량 시험 대비 요약집 — 확장판

객관식/단답식 대비: 개념 · 공식 · 함정 · 코드 스니펫 총정리

1. AI Foundations (수학 · 통계 · 데이터 전처리)

1.1 가설검정 & 신뢰구간

- 1 용어: 귀무가설(H_0 : 차이/효과 없음), 대립가설(H_1). $p\text{-value} < 0.05 \rightarrow H_0$ 기각(유의).
- 2 검정: t-검정(평균), χ^2 -검정(범주 분포), ANOVA(3그룹 이상 평균비교), 대응표본 t-검정(전후 비교).
- 3 95% 신뢰구간(CI): 표본 반복시 95% 구간이 모평균을 포함한다는 빈도주의 해석.
- 4 단답 핵심: ' $p=0.03$ '은 H_0 하에서 데이터가 관찰될 확률이 낮다는 뜻이지, H_0 가 참일 확률이 3%라는 뜻이 아님().

1.2 공분산/상관 & 분포

- 1 공분산은 방향만, 상관계수($-1 \sim 1$)는 선형강도까지. 비선형 의존성은 포착 못할 수 있음.
- 2 왜도(비대칭) · 첨도(꼬리 두꺼움)도 물어볼 수 있음: 정규분포는 왜도 ≈ 0 , 첨도 ≈ 3 (초과첨도 0).
- 3 정규/이항/포아송 분포 특성: 평균=분산(포아송), 이항 $\text{var}=np(1-p)$, 중심극한정리(CLT).

1.3 차원축소 (PCA/SVD)

- 1 목표: 분산을 최대 보존하는 직교 축으로 투영(라벨 미사용).
- 2 계산: 공분산행렬 고유값분해 또는 $X=SVD=U\Sigma V^T \rightarrow$ 주성분= V 열벡터, 분산 기여율 $\propto \Sigma^2$.
- 3 해석 포인트: 스케일링(표준화) 여부가 주성분에 큰 영향. 데이터 누출 방지 위해 train 스케일러로만 transform.

1.4 회귀 & 정규화

- 1 OLS 폐쇄형해: $\hat{\beta} = (X^T X)^{-1} X^T y$. 가정: 선형성, 등분산성, 독립성, 정규성(추론 시).
- 2 정규화: Ridge(L2, 가중치 제곱), Lasso(L1, 희소성 유도), ElasticNet(혼합).
- 3 편향-분산 트레이드오프: 정규화 $\uparrow \rightarrow$ 분산 $\downarrow$ · 편향 $\uparrow$. 최적점은 검증오차 최소 지점.

1.5 데이터 불균형 · 결측 · 전처리

- 1 불균형 대처: Class weight, 임곗값 튜닝(ROC $\rightarrow$ PR로 평가), Over/Under-sampling, SMOTE.
- 2 결측치: MCAR/MAR/MNAR 구분. 단순대치(mean/median), KNN/다중대치. 학습/검증 일관성 유지.
- 3 전처리 함정: 스케일러/임퓨터는 반드시 train으로 fit $\rightarrow$ train/test에 동일 transform. 누출 금지.

2. AI Modeling (모델 구성 · 학습 · 평가)

2.1 활성화함수 · 손실함수 매칭

- 1 Sigmoid: 이진확률, 기울기 소실 주의. ReLU: 간단 · 빠름, Dead neuron(음수영역) 가능 → Leaky/ELU로 보완.
- 2 Softmax+Cross - Entropy(CE): 다중분류 표준. 로짓 z 에서 $p=\text{softmax}(z)$, $L = - \sum y \cdot \log(p)$.
- 3 회귀: MSE/MAE(이상치 강건성은 MAE가 우수). Huber loss는 절충안.

2.2 최적화 · 스케줄 · 초기화

- 1 SGD vs Adam/AdamW(가중치 감쇠 분리). 일반론: Adam으로 탐색→SGD로 미세조정이 통용.
- 2 학습률 스케줄: Step/Cosine/OneCycle. 발산 시 $lr \downarrow$, 수렴 느릴 때 $lr \uparrow$ 또는 warmup.
- 3 가중치 초기화: Xavier(탄젠트/시그모이드), He(ReLU 계열).

2.3 과적합 제어

- 1 가중치 정규화(L1/L2), Dropout($p \approx 0.1 \sim 0.5$), 데이터 증강, Early Stopping(patience).
- 2 정규화 레이어: BatchNorm(배치 통계), LayerNorm(특징별), 목적은 내부 공변량 변화 완화 · 안정화.
- 3 배치크기: 크면 안정 · 일반화 저하 가능, 작으면 잡음 $\uparrow$ · 일반화 개선 가능(상황 의존).

2.4 평가 지표 & 혼동행렬

- 1 Accuracy 함정: 불균형일 때 왜곡 가능 → Precision/Recall/F1 우선.
- 2 ROC - AUC vs PR - AUC: 극단적 불균형은 PR - AUC가 민감.
- 3 단답 예: $F1 = 2PR/(P+R)$. Macro vs Micro 평균 차이(클래스 불균형 시 Macro로 클래스별 균등 가중).

3. AI Engineering (환경 · 코드 · 재현성)

3.1 환경/의존성 관리

가상환경 생성/활성화:

```
python -m venv .venv
```

```
source .venv/bin/activate # (Windows: .venv\Scripts\activate)
```

의존성 고정:

```
pip install -r requirements.txt
```

```
pip freeze > requirements.txt
```

재현성 팁: python/torch/cuda 버전 명시, random seed 고정(torch, numpy, random).

3.2 PyTorch 기본 학습/추론 템플릿

학습 루프(요지):

```
model.train(); optimizer.zero_grad();
```

```
out = model(x_batch); loss = crit(out, y_batch);
```

```
loss.backward(); optimizer.step()
```

추론:

```
model.eval()
```

```
with torch.no_grad():
```

```
y_pred = model(x_val)
```

체크포인트(최고 성능 저장):

```
if val_score > best: torch.save(model.state_dict(), 'best.pt')
```

3.3 Scikit - learn 파이프라인 & 직렬화

```
from sklearn.pipeline import Pipeline
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([('scaler', StandardScaler()), ('clf', LogisticRegression(class_weight='balanced'))])
```

```
pipe.fit(X_train, y_train) # 누출 방지: split→fit→transform
```

모델 저장/로드:

```
import joblib
```

```
joblib.dump(pipe, 'model.pkl'); pipe = joblib.load('model.pkl')
```

3.4 재현성 · 디버그 체크리스트

- 1 데이터 누출 여부(스케일/임퓨트/피쳐선택이 train에만 fit 되었는가).
- 2 검증전략(Stratified K - Fold, 시계열은 TimeSeriesSplit).
- 3 로그/실험기록: seed, 하이퍼파라미터, 버전메모.
- 4 GPU/메모리 오류시: 미니배치 축소, grad_accumulation, no_grad/half - precision(FP16) 검토.

4. 합정 정리 & 단답 대비 미니 Q/A

4.1 합정 Top 10

- 1 PCA와 LDA 차이: PCA(비지도, 분산 최대) vs LDA(지도, 클래스 분리 최대).
- 2 Dropout은 추론 시 꺼야 함(model.eval()).
- 3 BatchNorm은 배치 통계를 사용, 전체 데이터 정규화가 아님.
- 4 정규화(L1/L2)는 학습 손실에 패널티로 추가, 검증에서는 적용 X.
- 5 Accuracy 맹신 금물: 불균형이면 PR - AUC/F1을 확인.
- 6 Train/Test 분리 전에 스케일 fit 하면 누출.
- 7 Cross - Entropy는 확률적 해석(로그우도). MSE는 분류에서 경사 해상도 낮음.
- 8 이상치가 많을 때 MAE/Huber가 안정적.
- 9 k - Fold는 데이터 셔플 · 층화 확인. 시계열은 순서 보존.
- 10 정규화 강도↑는 분산↓ · 편향↑(트레이드오프).

4.2 단답 대비 25문 25답(핵심 한 줄)

- 1 Q1. p - value의 정의? → H_0 참 가정 하 관측 통계량 이상이 나올 확률.
- 2 Q2. 95% CI 의미? → 반복 표본에서 95%의 구간이 모수를 포함.
- 3 Q3. CE가 분류에 적합한 이유? → 로그우도로 확률적 학습, 경사 해상도 높음.
- 4 Q4. Softmax 출력 해석? → 각 클래스의 조건부 확률(합=1).
- 5 Q5. F1이 유용한 상황? → 불균형 데이터에서 정밀도/재현율 균형 평가.
- 6 Q6. Macro vs Micro F1? → Macro는 클래스별 평균, Micro는 전체 집계.
- 7 Q7. L1의 효과? → 희소성 유도 · 피쳐 선택.
- 8 Q8. L2의 효과? → 가중치 폭발 억제 · 안정화.
- 9 Q9. He 초기화가 어울리는 활성화? → ReLU 계열.
- 10 Q10. BatchNorm 목적? → 내부 공변량 변화 완화 · 학습 안정화.
- 11 Q11. Early Stopping 기준? → 검증 오차/점수 개선 정체 시 중단.
- 12 Q12. 불균형 임계값 조정 이유? → 재현율/정밀도 트레이드오프 제어.
- 13 Q13. ROC vs PR - AUC 선택? → 극단적 불균형은 PR - AUC.
- 14 Q14. 데이터 누출 예시? → 전체 데이터로 스케일 fit 후 분리.
- 15 Q15. OLS 가정 위반 시 대안? → 로버스트 회귀/가중 회귀/변환.
- 16 Q16. MAE vs MSE 차이? → MAE는 이상치 강건, MSE는 큰 오차에 민감.
- 17 Q17. CE와 NLL 관계? → CE는 softmax+NLL과 동치.
- 18 Q18. AdamW 장점? → weight decay를 모멘트 업데이트와 분리.
- 19 Q19. 과적합 징후? → train↑ val↓ 격차 확대.
- 20 Q20. 편향 - 분산 정의? → 단순화 오차 vs 데이터 민감 오차.
- 21 Q21. Stratified K - Fold 쓰는 이유? → 클래스 비율 유지.
- 22 Q22. 정규화와 표준화 차이? → [0,1] 스케일 vs 평균0 · 분산1.
- 23 Q23. KNN 임퓨테이션 주의? → 스케일 · 이웃 수 · 시간 비용.
- 24 Q24. Huber 손실 장점? → 이상치에 강건(δ 내 MSE, 밖 MAE).
- 25 Q25. 모델 저장 시 권장? → 최고 성능 기준 체크포인트+메타(버전/seed) 기록.

5. 시험 직전 체크리스트(단답/객관 대비)

- 1 정의형 한 줄 암기(위 25문).
- 2 공식 암기: F1, CE, OLS, ROC/PR 개념, PCA 절차.
- 3 코드 키워드: `model.eval()`, `torch.no_grad()`, `class_weight`, `Pipeline`, `joblib`.
- 4 함정 회피: 전처리 누출, Dropout 추론시 비활성, 불균형에서 Accuracy 맹신 금지.
- 5 시간전략: 쉬운 객관식→단답 짧게→남은 시간에 계산형/헛갈리는 문항 재검토.

※ 본 요약은 객관식 · 단답식에 최적화. 예제문항 수렴 시, 동일 포맷으로 해설집 추가 가능.